



Somos  
F5

# Construyendo el motor del Dato

Creación de una API y conexión con base de datos

ABRIMOS RUTAS **INCLUSIVAS**  
AL TALENTO DIGITAL

# Hoja de ruta detallada

|                                                         |                                                                                                                                        |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bloque I: Cambio de paradigma</b>                    | <ul style="list-style-type: none"><li>• Arquitectura cliente-servidor</li><li>• El rol de proveedor de servicios</li></ul>             |
| <b>Bloque II: Repasamos la anatomía de una API REST</b> | <ul style="list-style-type: none"><li>• Protocolo HTTP: Verbos y Códigos de Estado</li><li>• Endpoints y semántica</li></ul>           |
| <b>Bloque III: Lógica y persistencia</b>                | <ul style="list-style-type: none"><li>• Asincronía: Async/Await.</li><li>• ORM: SQLAlchemy</li><li>• Motor BBDD: PostgreSQL.</li></ul> |
| <b>Bloque IV: Buenas prácticas</b>                      | <ul style="list-style-type: none"><li>• Validación con Pydantic.</li><li>• Modularidad y estructura de archivos.</li></ul>             |



# De consumidores a proveedores

---

## El Cambio de Paradigma

- **Antes (Consumidor):** Vuestra app solicita datos a una API externa (ej. Twitter, Weather API).
- **Ahora (Proveedor):** Vuestra app es la fuente de verdad. Otros sistemas (o vuestro propio Frontend) os pedirán datos a vosotros.

## Arquitectura Cliente-Servidor

- **Cliente:** Terminal, Navegador, App móvil (pide recursos).
- **Servidor (Vuestra API):** Escucha, procesa lógica, consulta base de datos y responde.



# Actividad: Trazando la ruta del dato

**El Caso:** Un usuario entra en vuestra futura aplicación y hace clic en "Crear Nueva Tarea".

**Instrucciones:** Ordenad los siguientes elementos cronológicamente desde el clic del usuario hasta el guardado físico, y describid en una frase qué ocurre en cada paso.

## Elementos a ordenar:

- Motor de Base de Datos (PostgreSQL)
- Navegador Web (Frontend)
- Framework Web (FastAPI)
- Red / Protocolo HTTP
- ORM (SQLAlchemy)



# Protocolo HTTP y Verbos REST

## Los Verbos del Backend (Operaciones CRUD)

| Acción CRUD | Verbo HTTP         | Propósito                                   |
|-------------|--------------------|---------------------------------------------|
| Create      | <b>POST</b>        | Crear un nuevo registro (ej. nueva receta). |
| Read        | <b>GET</b>         | Obtener información (ej. lista de recetas). |
| Update      | <b>PUT / PATCH</b> | Modificar un registro existente.            |
| Delete      | <b>DELETE</b>      | Eliminar un registro de la base de datos.   |



# Códigos de Estado: La API "habla"

## ¿Qué nos dice el servidor?

- **2xx (Éxito):** 200 OK, 201 Created (¡Importante tras un POST!).
- **4xx (Error del Cliente):** 400 Bad Request, 404 Not Found (El recurso no existe).
- **5xx (Error del Servidor):** 500 Internal Server Error (Algo se rompió en nuestro código).

**Un buen backend nunca responde 200 si ha habido un error.**

**La honestidad de la API es clave.**



# Asincronía: Cómo evitar bloquear la puerta

## El Bloqueo de Input/Output

En el backend, las tareas más lentas no son los cálculos, sino esperar a la Base de Datos.

```
@app.get("/items")
async def read_items():
    # El servidor "pausa" esta petición aquí y atiende a otros clientes
    # mientras la base de datos busca la información.
    data = await database.fetch_all("SELECT * FROM items")
    return data
```

- `async`: Declara que la función es asíncrona (una corrutina).
- `await`: Cede el control de vuelta al servidor mientras espera un resultado lento.



# La capa de persistencia: **SQL + ORM**

● Escribir sentencias SQL puras como strings en Python es propenso a errores y vulnerable a ataques (Inyección SQL). Para evitarlo, usamos un **ORM (Object-Relational Mapping)**.

- **Motor de base de datos:** MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, SQLite...
- **Librería ORM:** Facilita la interacción con bases de datos SQL utilizando objetos de Python, evitando escribir SQL directo. Usaremos SQLAlchemy.

¿**Por qué usarlo?** Nos permite interactuar con tablas usando clases y objetos de Python en lugar de escribir SQL puro (Strings).

- *SQL:* `UPDATE users SET name = 'Andrea' WHERE id = 1;`
- *Python:* `user.name = "Andrea"`





# Validación con Pydantic

## Pydantic: El guardián de la API

Antes de que un dato toque nuestra Base de Datos (SQLAlchemy), debe ser validado. Ejemplo:

- Si recibimos "edad": "veinte" en lugar de un número, el sistema debe rechazar la petición antes de procesarla.

**Pydantic** utiliza las anotaciones de tipo de Python para definir "Esquemas" (**Schemas**) estrictos de validación.

Si el cliente envía un JSON que no cumple este contrato, FastAPI y Pydantic devuelven automáticamente un error HTTP 422 (Unprocessable Entity).

```
from pydantic import BaseModel, EmailStr

# Esquema de Entrada (Lo que el cliente debe enviar en el POST)
class UserCreate(BaseModel):
    username: str
    email: EmailStr
    age: int
    is_active: bool = True
```

# Actividad: Validación con Pydantic

**Contexto:** Habéis programado el esquema **UserCreate** y un cliente Frontend os hace una petición POST enviando este JSON en el cuerpo del mensaje:

```
{  
  "username": "admin_f5",  
  "email": "usuario.f5@com",  
  "age": "25",  
  "is_active": "yes"  
}
```

```
from pydantic import BaseModel, EmailStr  
  
# Esquema de Entrada (Lo que el cliente debe enviar en el POST)  
class UserCreate(BaseModel):  
    username: str  
    email: EmailStr  
    age: int  
    is_active: bool = True
```

**Misión:** Analizad el JSON basándoos en el esquema Pydantic.

- ¿Qué responderá la API?
- ¿Se guardará en la base de datos o fallará?
- Enumerad los posibles problemas.



# Modularidad: Organizando el caos

## Estructura de un proyecto profesional

Separar responsabilidades para facilitar el testeo y mantenimiento:

- `main.py`: El punto de entrada. Define rutas y arranca FastAPI.
- `database.py`: La "fontanería". Conexión y sesión de la DB.
- `models.py`: El espejo de la base de datos (SQLAlchemy).
- `schemas.py`: Los contratos de datos (Pydantic).



# Proyecto Individual

---

## De la Teoría a la Práctica

En las próximas sesiones programaréis vuestra propia API REST individual:

- **Recurso a elegir:** Pensad en algo que os interese a nivel personal. Tareas, Videojuegos, Recetas,...
- **Requisitos:**
  - Operaciones CRUD completas.
  - Persistencia en PostgreSQL.
  - Documentación automática (Swagger/OpenAPI).
  - Modularidad estricta.

